

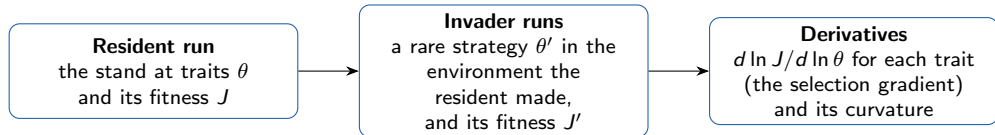
Fitness gradients for TF24 in plant

What changed in plant, odelia and phylloptim, what it costs and what it buys

8 October 2026

An evolutionary analysis needs fitness and its derivatives, many times over

regnans finds and classifies singular strategies from plant runs of three kinds.



- One singular strategy in one trait takes about **84 such runs**. A resident run on a 40-year record takes about 90 s.
- Until now plant returned J alone, so every derivative came from repeated runs.

Run counts: docs/design-grid-controller.md, "Where the arc starts".

The whole analysis step, end to end, in plant's R interface

```
p    <- add_strategies(scm_base_parameters("TF24"), trait_matrix(0.32, "lma"))
p$node_schedule_times <- list(seq(0, 39.6, length.out = 108)) # 108 introductions over 40 years

base <- control_tf24(3e-5, Control(node_density_in_birth_date = TRUE,      # parts 1, 5, 6, 8
                                   ode_split_sign_changes = TRUE))        # part 4
pilot <- run_scm(p54, env, control_tf24(1e-3), record_trajectory = TRUE) # p at 54 introductions
ctrl  <- control_window(pilot, list(lma_half, lma_double), base = base)  # part 7

scm <- run_scm(p, env, ctrl, record_trajectory = TRUE)
g    <- stand_gradient(scm, "offspring_production") # all 49 derivatives in one pass, part 2
scm$run_mutant(invader)                          # an invader on the resident's steps, part 3
d    <- diagnose_scm(p, env, ctrl, invaders)        # error estimate and failures, part 9
```

On the 40-year long-drought record the middle block takes **209 s**: 37 s for the pilot, 40 s for the resident run and 132 s for its gradient. It returns J and all 49 elasticities. The same answers by brute force take 886 s. Every function above is on the branches. The next slide lists the nine parts, and the deck takes them in order.

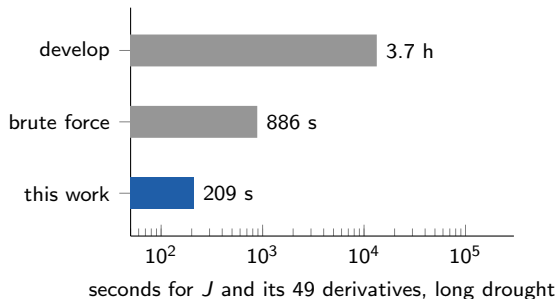
Timed alone, one run each: docs/measurements/headline/.

What was built: nine parts

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

Part 1 makes the birth-date coordinate's integrals exact. Parts 2 and 3 make gradients possible, and part 4 makes them smooth in the traits. Parts 5 to 7 make each run cheaper, and parts 8 and 9 let a run say how accurate it is. Each part closes one issue on the aornugent forks and is off by default.

Altogether: the same answers in a quarter to a half of brute force's time



develop plant's develop with its default Control1. One run takes 134 s; its gradient needs 100 runs by differences, so this bar is computed.

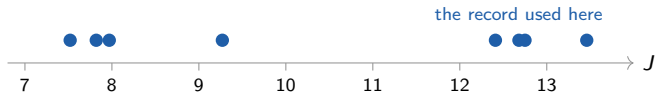
brute force this work's code with parts 4 to 7 off: a tight tolerance ($1e-5$) and twice the introductions (215).

this work the code on slide 3, pilot included.

- On the episodic record the ratio is also 0.24 (128 s against 536 s), and all 50 quantities lie within 0.40ε of brute force's. The next two slides define ε .
- On long drought 108 introductions are too few, and 215 match brute force's accuracy at 0.43 of its time. The table is near the end.

How close is close enough? The weather sets the scale

Eight 40-year rainfall records were drawn from one long-drought climate. They differ only in the day-to-day weather. The same stand, run on each, earns these fitnesses J :



- Nothing about the stand changed between these runs, yet J ranges from 7.5 to 13.5. In logs, their standard deviation is **0.25**.
- A numerical error far below that spread cannot change what an analysis concludes about this climate. One of comparable size could.
- So the tolerance for $\ln J$ is set at a tenth of the spread: $\varepsilon = 0.025$, or 2.5% of J .

Each derivative gets its own tolerance by the same rule

The same eight records give each elasticity a spread too. Its ε is a tenth of its standard deviation, and never below 0.01.

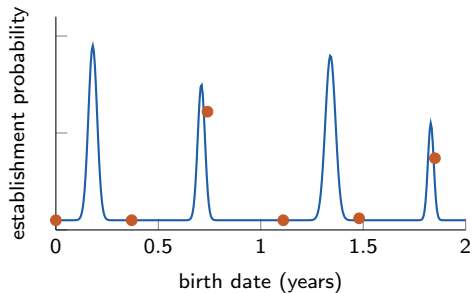
quantity, for the resident	mean over 8 records	standard deviation	ε
$\ln J$	2.32	0.25	0.025
elasticity of J in <code>lma</code>	-9.6	0.87	0.087
elasticity of J in <code>a_dG2</code>	1.10	0.19	0.019

- The rest of the deck reports errors as multiples of ε . “Within 0.4ε ” means within 0.4 of each quantity’s own tolerance.
- *Accurate* means within ε of the converged answer.
- *Reproducible* means a 5% change of tolerance, or moving every introduction by a quarter of its spacing, moves each quantity by less than $\varepsilon/3$.
- Both are checked on five records: constant rain, long-wet, episodic, dry and long drought.

Part 1 of 9: establishment integrated exactly between cohorts

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The problem: establishment changes within days, cohorts are months apart



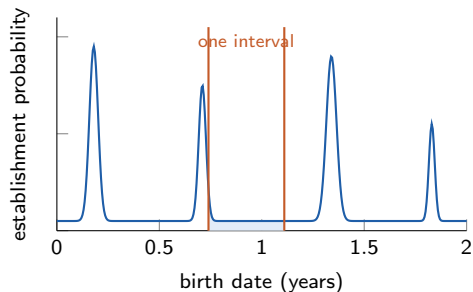
Shape for illustration.

plant#94 (PLANT-93).

On the birth-date coordinate each cohort stands for the seedlings born between it and the next cohort. Its density was the birth rate times the establishment probability at its own birth date (the orange points), and integrals over birth date were trapezia through those samples.

TF24's establishment rises after rain and falls as the soil dries, within days. Cohorts are introduced weeks to months apart, so the samples catch some pulses and miss others. J then depended on *when* cohorts were introduced, as well as on how many there were.

How it works: each cohort carries the establishment of its interval as a state



Shape for illustration.

1. While a cohort is the newest, it integrates the establishment probability over its interval as an ODE state, together with that integral's first moment. The solver's own steps resolve each pulse.
2. When the next cohort is introduced, the interval closes and its integrals stop changing.
3. An integral over birth date weights each cohort by establishment integrated against its hat function. Only the rest of the integrand, which is smooth, is interpolated between cohorts.

A cohort's density is then no longer a state of its own.

The result: establishment is exact, whatever the introductions

- The establishment term is integrated to the solver's tolerance, however the cohorts fall relative to the rain. What remains of the node error comes from the smooth part of the integrand.
- Before, a run's error estimate from its nodes' values saw only 4–5% of the true error: seedlings born into a closed establishment window made steps between nodes that no nodal value shows. Parts 8 and 9 rely on that cause being gone.
- The height coordinate is unchanged, and K93, which establishes every seed, gives the same answers to ten digits.
- On this coordinate `Species$log_densities_state` is removed and `refine_schedule()` raises an error.

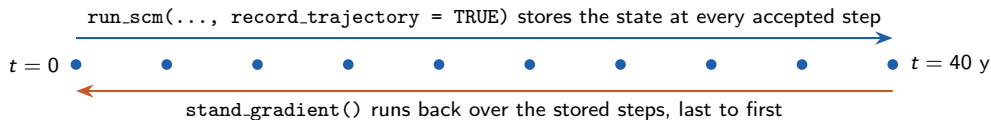
Part 2 of 9: every derivative from one reverse pass

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The problem: a gradient by differences costs two runs per parameter

- A central difference runs the stand at $\theta_j(1 \pm h)$ for each parameter j . TF24 has 49 differentiable parameters, so one gradient costs about **98 runs**.
- An analysis needs a gradient for the resident and one for each invader, at every step of its search.
- Where moving a trait moves an event, such as a cohort reaching maturity between two steps, the difference does not converge as h shrinks.

How it works: the run records its steps, and one pass runs back over them



At each stored step, from the last to the first, the pass:

1. re-evaluates the step's six Runge–Kutta stages, with XAD recording the arithmetic;
2. takes λ , the derivative of $\ln J$ in the state at the step's end, and turns it into the derivative in the state at the step's start;
3. adds that step's contribution to $\partial \ln J / \partial \theta$ for all 49 parameters at once.

Where a cohort was introduced, the pass carries λ into the newborn's initial size, so seed traits get their share. The leaf's inner solve is never recorded: `phyloptim` supplies its derivatives directly. Only doubles cross into R.

The result: all 49 derivatives for the cost of 2.5 runs

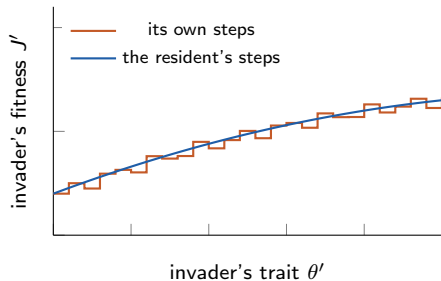
- One pass costs **2.5–2.7 runs**, whatever the number of introductions, against about 98 runs by differences.
- It agrees with central differences to $7e-4$ of the resident's `lma` elasticity, and closer for the other entries checked.
- A derivative that does not exist is refused with a stated reason; a NaN without one would be a bug.

`docs/measurements/perf-adjoint.md`; `eps-spread.md`.

Part 3 of 9: invaders run on the resident's recorded steps

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	<i>J</i> depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The problem: an invader's own steps made its fitness jump



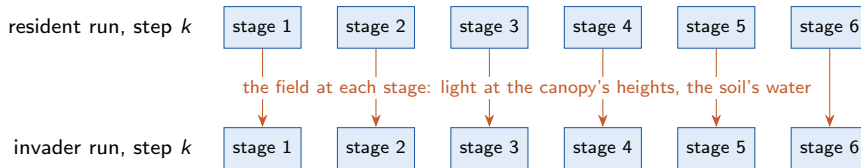
Shape for illustration.

An invader is a rare strategy, so it does not change the light or water the resident made. Before, plant still integrated it with step sizes of its own choosing.

Each small change in θ' could change those step sizes, and each change of steps moved J' by a small, unrelated amount. Differences of J' , which is how a selection gradient was taken, picked up those jumps.

And at the resident's own traits, J' came out slightly different from J .

How it works: the invader reads the field the resident recorded



- The resident's run records every accepted step: its time and size, and at each of its six stages, the field every cohort read.
- `scm$run_mutant(invader)` integrates the invader's cohorts on exactly those steps. At each stage it loads the recorded field instead of building one.
- The invader's own reverse pass holds that field fixed, which is what a selection gradient asks for.

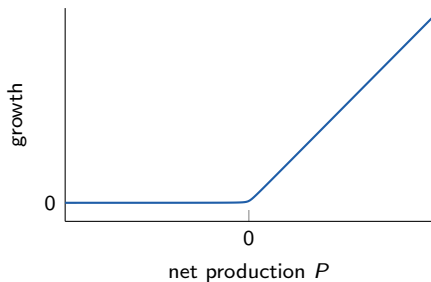
The result: J' is smooth in θ' , and equals J at the resident's traits

- With the steps fixed, J' is a smooth function of θ' , so its differences and its reverse pass are meaningful.
- An invader at the resident's own traits returns $J' = J$ to the last bit. The tests check this on every record.
- The first invader reuses the resident's recorded run instead of repeating it: 139 s against 300 s.
- An invader run costs about 0.77 of a resident run.

Part 4 of 9: a cohort's step cut where its net production changes sign

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The problem: growth has a corner where net production crosses zero

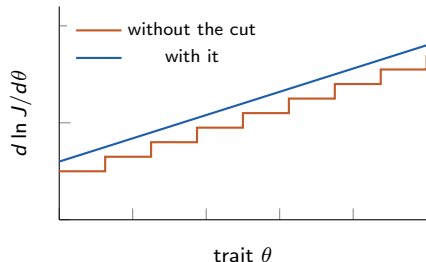


TF24 grows a cohort by the positive part of its net production P . Below zero a cohort draws on storage and does not grow.

The positive part is smoothed over a band of 10^{-4} kg/yr. A cohort crosses that band in a small fraction of one step: in the median crossing the step is 5×10^4 times longer than the bend.

So at the scale of a step, the cohort's rate has a corner.

The problem: a corner inside a step turns the gradient into a staircase



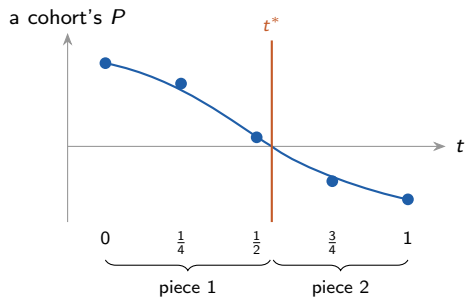
Shape for illustration.

A Runge–Kutta step evaluates the rate at six stages inside the step. Its error from a corner depends on where the corner sits among those stages.

As θ moves, each crossing slides through the step. Each time it passes a stage the error jumps, so the gradient moves in steps. A 5% change of tolerance moves every step boundary, and so moves the gradient by up to 0.52ε .

A curvature taken across one of those jumps was off by 13ε .

How it works: find the crossing inside the step, and integrate that cohort in two pieces



After odelia accepts a step, it hands plant the step just taken, with an interpolant of the whole state across it. With `ode_split_sign_changes = TRUE`, plant then:

1. reads each cohort's P at five fractions of the step, $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}$ and 1 , and finds where the curve through them crosses zero;
2. for each cohort that crosses, integrates that cohort alone from the step's start to t^* and from t^* to the end, in the light and water read off the interpolant;
3. leaves every other cohort and the soil on the whole step.

How it works in reverse: the cut moves with the traits

- In the reverse pass, each piece is a step of its own: its stages are re-evaluated and recorded like any other step's.
- The cut time t^* is where $P(t^*; \theta) = 0$, so it moves as θ moves. The pass differentiates it through that equation (the implicit function theorem), so $dt^*/d\theta$ enters the gradient.
- plant owns which cohort is cut, the field it reads and its pieces. One body of code serves the forward run and the reverse pass, so the two cannot disagree about a cut. odelia owns the arithmetic: the interpolant, the pieces and the root-finding.

The result: smooth, reproducible gradients for 5–9% more time

- $\ln J$ lies 3–13 $\times$ nearer the converged answer wherever cohorts cross zero.
- A 5% change of tolerance moves no gradient entry by more than $\varepsilon/6$, against up to 0.52ε before.
- The reverse pass through the cuts agrees with central differences to $2e-3$ on every record.
- The leaf's solve now stops at roundoff (phylloptim#17), so re-running a step at a trait 10^{-12} away moves $\ln J$ smoothly. Its noise fell from $2.7e-8$ to about $1e-14$.
- The cuts cost 5–9% of a gradient run.

Part 5 of 9: TF24's step settings in one preset

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

How plant decides a step is accurate enough, and what a weight does

plant's integrator accepts a step when, for every state y_i , its estimated error is below its allowed error:

$$|\text{error}_i| \leq w_i (\text{tol_rel} |y_i| + \text{tol_abs})$$

- w_i is a weight per state, new in this work and 1 by default. A weight of 10 lets that state carry ten times the error, so it takes fewer, longer steps.
- Each weight can also change over the run: part 7 raises them late in the patch's life.
- `control_tf24(tol)` sets TF24's values for these knobs once, so every run of an analysis uses the same ones. plant's `Control()` defaults do not change.

```
odelia OdeControl::adjust_step_size; state-weights.
```

What `control_tf24()` sets, and why each value

setting	why
<code>tol_abs = 10⁻⁴ × tol_rel</code>	Tied this way, <i>J</i> 's error stays within about half of <code>tol</code> . With plant's default absolute tolerance it was about 20× larger at <code>tol = 1e-4</code> .
soil layers' weight 10	The soil set most steps while carrying little of <i>J</i> 's error. With ten times its error allowed, a gradient run takes 21% less time, and both roles still pass the accuracy and reproducibility tests at 3e-5.
every weight bounded at 100	Part 7 multiplies weights by up to 100. Unbounded, the soil's reached 1000, an accepted step could carry a soil layer to its potential ceiling, and the gradient was refused there.
steps capped at 15 days	An invader's storage pools go unstable on steps longer than 26 days. On the episodic and dry records, the invader at twice the resident's <code>lma</code> failed on 27-day steps.
soil alone below a 10% share	Part 6.

With part 7, these settings save 31–34% of a gradient run against the tied tolerance alone, and with the cap no invader fails on any record.

Part 6 of 9: the soil stepped on its own

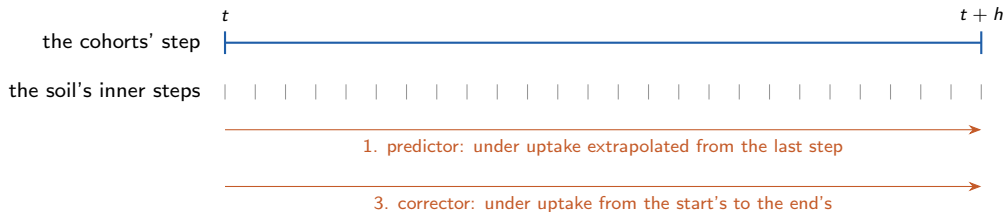
	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The problem: the soil's stability sets almost every step

- Water moves between TF24's soil layers within days. An explicit Runge–Kutta step longer than that makes the soil unstable, so the step must stay short.
- On long drought the soil alone forces **16 447 of the run's 17 684 steps**. The cohorts on their own would need about 9200.
- For much of a run the stand draws only a small share of the water moving through the soil, so the soil hardly depends on the plants over a single step.

docs/grid-dynamics.md §1; docs/design-grid-controller.md, "The time axis".

How it works: the soil takes short inner steps inside each step



At a step's start plant computes the uptake's share of the water moving through the soil. If it is under `ode_soil_alone_share` (0.1 in the preset), `odelia`:

1. integrates the soil alone across the step in short inner steps, under the plants' uptake extrapolated from the last step;
2. takes the cohorts' Cash–Karp step, each stage reading the soil from step 1;
3. integrates the soil again, under the uptake the step found, and keeps that end;
4. counts the gap between the two soil passes in the step's error.

The run records the extrapolation and the inner steps, so a replay and the reverse pass take the same steps.

The result: a gradient run takes 0.63 of the time

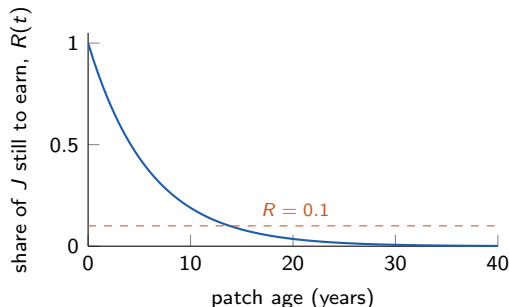
- Timed alone, a gradient run takes 376 s against 596 s.
- Steps fall 42–45% on the pulsed records and 64% under constant rain.
- Every elasticity moves by less than 0.063ϵ , and $\ln J$ stays within $5.8e-5$ of a reference at tolerance $1e-6$ on all five records.
- It replaces an implicit solver for the soil (ARK) that was built first: under constant rain it costs 0.97 of that solver's steps, and the ARK branches are retired.

docs/measurements/soil-alone/prereg.txt, gates G1–G7.

Part 7 of 9: tolerance loosened once fitness is earned

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The idea: late in a patch's life there is little fitness left to get wrong

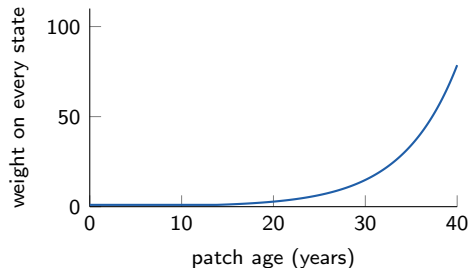


Shape for illustration.

Offspring are produced mostly early in a patch's life. Write $R(t)$ for the share of the run's J still to be earned after time t .

A step late in the run can change J only through what is still to be earned. Spending the same tolerance there as early on buys accuracy J does not need.

How it works: a pilot run reads $R(t)$, and the weights follow it



Shape for illustration.

`control_window(pilot, invaders, base):`

1. runs nothing new: the caller's pilot is the stand at 54 introductions and tolerance $1e-3$, with its recorded trajectory;
2. walks each invader on it, here `1ma` at half and twice the resident's;
3. reads $R(t)$ at each recorded step, for the stand and each invader, and keeps the largest;
4. sets every state's weight to $0.1/R(t)$, between 1 and 100.

Weights stay at 1 until only a tenth of J is left to earn.

PLANT-106 (plant#106); docs/measurements/window-pilot/.

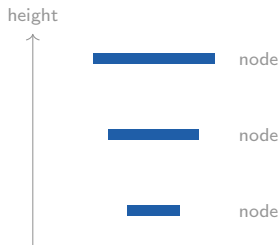
The result: about a fifth fewer steps on the pulsed records

- The weights save 21–23% of a run's steps on the pulsed records, and 2% under constant rain, where J is earned more evenly.
- The pilot costs 0.4–0.8 of the run it serves. One pilot serves every run of an analysis around one resident.
- On all five records nothing fails, and $\ln J$ lies within $5.8\text{e-}5$ of a reference at tolerance $1\text{e-}6$.

Part 8 of 9: leaf area spread between nodes

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

The problem: the light field stepped at every node, so a run could not estimate its error

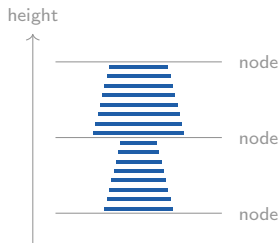


On the birth-date coordinate, each node stands for the cohorts born between it and its neighbours. plant put all of that leaf area at the node's own height, one crown per node.

The light field then stepped at each node's height, and its error did not fall at any fixed rate as nodes were added. Over 54, 108 and 215 introductions the ratio of successive changes was 8.5 on long-wet and 6.6 on long drought, and for 15–18 quantities the changes switched sign.

A run at half the introductions reported only 0.33–0.41 of the true error.

How it works: eight point crowns between each pair of nodes



Each interval between two neighbouring nodes now spreads its leaf area over eight point crowns, at evenly spaced heights between the two nodes' heights.

- Each crown carries both nodes' shares of the interval, each node's share weighted toward its own end.
- At the ground every crown is whole, so the field there is unchanged.
- Where two cohorts cross in height, the crowns are summed tallest first, which keeps the light field's fast path (PLANT-107).

PLANT-107, PLANT-108 (plant#107, #108).

The result: the error falls fourfold per doubling, so it can be estimated

- With the leaf area spread, the error falls with the square of the spacing. Measured ratios of successive changes are 3.5–4.0, against 4 in theory.
- That makes the error computable from two runs. With Q a quantity from a run and $Q_{1/2}$ the same quantity at half the introductions,

$$\text{converged answer} \approx Q + \frac{Q - Q_{1/2}}{3}.$$

- The half-introduction run now reports 0.87–0.99 of the true error, against 0.33–0.41 before.
- The spread costs 6.7% of a gradient run.
- An invader can be walked on some of the run's introductions, matched by birth date (PLANT-109).

Part 9 of 9: each run reports its error and failures

	what was built	the problem it fixes	in plant
1	Establishment integrated exactly between cohorts	J depended on when cohorts were introduced	the birth-date coordinate
2	Every derivative from one reverse pass	98 runs per gradient by differences	<code>stand_gradient()</code>
3	Invaders run on the resident's recorded steps	an invader's fitness jumped as its own steps moved	<code>run_mutant()</code>
4	A cohort's step cut where its net production changes sign	gradients shaped like staircases	<code>ode_split_sign_changes</code>
5	TF24's step settings in one preset	wasted steps, refused gradients, failed invaders	<code>control_tf24()</code>
6	The soil stepped on its own	the soil's stability set 93% of the steps	<code>ode_soil_alone_share</code>
7	Tolerance loosened once fitness is earned	late steps spent accuracy on nothing	<code>control_window()</code>
8	Leaf area spread between nodes	a run could not estimate its own error	the birth-date light field
9	Each run reports its error and failures	no error estimate; one failure ended an analysis	<code>diagnose_scm()</code>

How it works: the run, repeated at every other and every fourth introduction



`diagnose_scm(p, env, ctrl, invaders)` runs the stand three times, keeping both ends, and walks each invader on each run. For every quantity ($\ln J$, $\ln J'$ and each elasticity) it returns:

- **error** = $(Q - Q_{1/2})/3$, the correction to the converged answer;
- **ratio** = $(Q_{1/2} - Q_{1/4})/(Q - Q_{1/2})$, near 4 where the estimate holds;

and, beside them, each invader's distance $\ln(\theta'/\theta)$ in each parameter it moves, and each run, invader or gradient that failed, with its message.

An example: three runs of a small test stand

An eight-year stand at 9, 5 and 3 introductions, with one invader at 1.5 times the resident's `hmat` and one that cannot be built:

run	quantity	value	half	quarter	error	ratio
resident	<code>ln J</code>	4.7033	4.7979	5.1101	-0.0315	3.30
resident	<code>lma</code>	-1.6028	-1.5512	-1.4264	-0.0172	2.42
late	<code>ln J</code>	5.5647	5.6787	6.0344	-0.0380	3.12

run	stage	message
broken	walk	<code>L.tip</code> must be shorter than the birth-size flow path...

- The ratios of 2.4–3.3 say nine introductions are too few to trust the estimate.
- The invader `late` is 0.405 away from the resident in `hmat`.
- The broken invader becomes three rows of failures, and the rest still answer. The two extra runs cost about three quarters of a run.

Altogether: on long drought, 215 introductions match brute force at 0.43 of its time

The reference is extrapolated from this work's runs at 108 and 215 introductions, $Q_{215} + (Q_{215} - Q_{108})/3$. Each row compares $\ln J$ and the 49 elasticities with it, in units of each quantity's ε . Time is relative to brute force's.

long drought	$\ln J$	elasticities, median	largest	time
brute force: 215 introductions, 1e-5	0.05ε	0.12ε	0.49ε	1
this work, 215 introductions	0.05ε	0.13ε	0.53ε	0.43
this work, 108 introductions	0.20ε	0.50ε	2.1ε	0.24

- At 108 introductions the remaining error comes from the nodes. The half-introduction run reports it, so a run can tell that it needs more introductions.
- On the episodic record 108 introductions suffice: every quantity lies within 0.40ε of brute force's.
- No run, invader or gradient fails on any of the five records.

What changes for code that uses plant today

- Every new behaviour sits behind a `Control` field or a new function, and is off by default. With it off, results are bit-identical, apart from the two moves below.
- The height coordinate, plant's default, does not move.
- **FF16 on the birth-date coordinate moves**, because part 8 changes how leaf area enters the light field. The one-species anchor goes from 17.14 to 18.01, and the two-species test from 12.05 and 16.59 to 12.07 and 16.76. At the default schedule FF16's error is now $9\text{--}25\times$ larger. It falls at the square law, so a half-introduction run estimates it. Each move was registered before the references were blessed again.
- TF24's model version goes to v13. Its numbers change with the reverse pass, a storage pool offset, and a newborn's height found to roundoff.
- Under plant's default `Control()`, TF24's J on long drought is 0.45 on `develop`, against 12.75 converged. On this work's code the same defaults give 0.18, so the default settings, more than the model, carry that gap.

Known gaps

- **Episodic rain needs finer introductions** for part 9's estimate to hold. Its error falls at the square law only once the spacing is below its dry spells (median 31 days), past 429 introductions.
- **Under constant rain the gradients move by up to 3ϵ** with where the later introductions sit; only the first year is graded finer.
- **A gradient blow-up with the soil stepped with the plants at a loose tolerance.** At tolerance $1e-3$, with the soil's weight at 10 and no inner steps, the resident's elasticities come out near 10^{50} while J is right, and nothing refuses. Part 6, now the preset's default, removes it at every tolerance tried. Not yet root-caused.
- **How far one resident's recorded steps can serve** a moved resident or a distant invader is unmeasured; `diagnose_scm()` reports the distance without a bound.
- The branches sit on the fork's `develop` with `odelia 0.5.0`, and need a rebase onto upstream `develop` (`odelia 0.6.2`) before PRs open there.

How to review: four groups, merged in this order

group	branches	read first	its test
Parts 1–3: counts and gradients	PLANT-93, offspring-adjoint, PLANT-95–99	stand_gradient, run_mutant	the reverse pass against central differences; $J' = J$ to the bit
Part 4: the cut	state-weights, PLANT-100–103, odelia#53–54, phylloptim#17	where a step is cut, and how the reverse pass crosses the cut	the 5% tolerance change; the pass against differences
Parts 5–7: step settings	PLANT-104–106, odelia#55	control_tf24, control_window	replays bit for bit; nothing fails on five records
Parts 8–9: nodes and diagnostics	PLANT-107–110	the spread in the light field; diagnose_scm	the square law; the FF16 re-bless

Each group is bit-identical with its new controls off. Part 4 is the largest change, and changes what odelia hands plant after a step. Part 8 holds the one decision for maintainers: whether the spread should apply to FF16's birth-date coordinate.